

# 41. Une rangée, puis un mur

Quatre-vingts appels en quatre lignes.

## 1. Ce qu'il faut comprendre

Écrire les quatre-vingts `poser()` à la main, ce sont quatre-vingts occasions de se tromper d'une case — et le jour où l'écran change de taille, il faut tout relire.

Deux boucles pour les bords horizontaux, deux pour les verticaux. Les coins sont posés deux fois : cela ne coûte rien, et l'écrire autrement demanderait quatre conditions.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **411 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         103 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte - attend de pouvoir écrire une case
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() - écrit un texte, lettre par lettre
CopierTuiles   58 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         96 o ← CopierTuiles - 5 lettres sur 43, et 1 tuile dessinée
"ENFERME"      7 o ← texte() - un texte gravé
```

Lettres gravées : EFMNR

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 4      49 o sur place, dans main() + AdresseCase (10 o)
texte() × 1      11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1       3 o sur place, dans main() + AttendreImage (28 o)
```

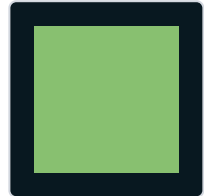
## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : dessine un mur tout autour de l'écran, et écrit ENFERME au milieu.
//
// Une tuile MUR : un bord sombre (#) et un intérieur moyen clair (-).
// Deux boucles for posent les murs : une pour le haut et le bas,
// une pour la gauche et la droite.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <poser>    // pose une tuile sur une case du fond
#include <texte>    // écrit un texte à l'écran
```

## 2. UN DESSIN — MUR

```
Tuile MUR = {
    "#####",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#-----#",
    "#####",
};
```



à l'écran

## 3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le jeu commence ici.
    for (uint8_t x = 0; x < 20; x++) { poser(x, 0, MUR); poser(x, 17, MUR); }
    // x = 0 à 19 : chaque colonne reçoit un mur
    // en haut (ligne 0) ET en bas (ligne 17).
    // Les accolades regroupent les deux poser() :
    // la boucle répète les deux.
    for (uint8_t y = 0; y < 18; y++) { poser(0, y, MUR); poser(19, y, MUR); }
    // y = 0 à 17 : chaque ligne reçoit un mur
    // à gauche (colonne 0) ET à droite (colonne 19).

    texte(6, 8, "ENFERME");    // Le mot au milieu.

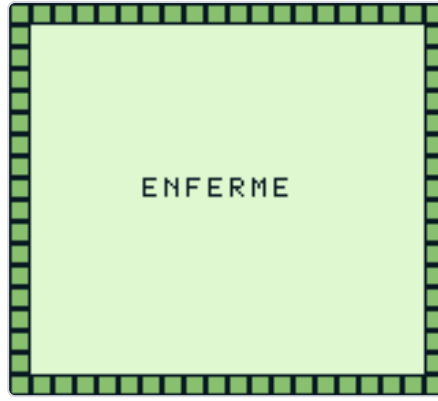
    while (true) {
        // Répète pour toujours :
        image();               // attend l'image suivante.
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

Un cadre tout autour de l'écran, et « ENFERME » au milieu.

## 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ les quatre coins sont posés
- ✓ le haut et le bas sont pleins
- ✓ les côtés aussi
- ✓ le milieu est resté libre

## 6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

[← l'atelier comp](#)

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

MUR

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

MUR

tuile n° 44 — poser(x, y, MUR

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

MUR ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

MUR peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

Ensuite : **42. Un damier** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++